AIGC 5005 | MODULE 3 | 3.5% OF FINAL GRADE

Lab 03: Refactor into a Package

Due date posted on Blackboard

Take the program you wrote for Lab 02 and restructure it as an installable Python package: records as classes, a family of related classes that share a base class, a clear module layout, and a conda environment that a classmate can rebuild from your repository alone. The Module 3 worked example did this to a library catalogue. This lab asks you to do it to your own code.

The program should do exactly what it did before. The same data goes in, the same summary comes out, and a failed download still ends with a clear message rather than a traceback. What changes is where each piece of the code lives, and how easy it is to find, reuse and change.

What this lab is about

Lab 02 was marked on whether the program was correct and robust. Lab 03 is marked on design. A script that works is a good start, but a project you can hand to someone else needs a shape: classes with one clear job each, modules named for what they contain, and an environment file that removes the phrase "it works on my machine".

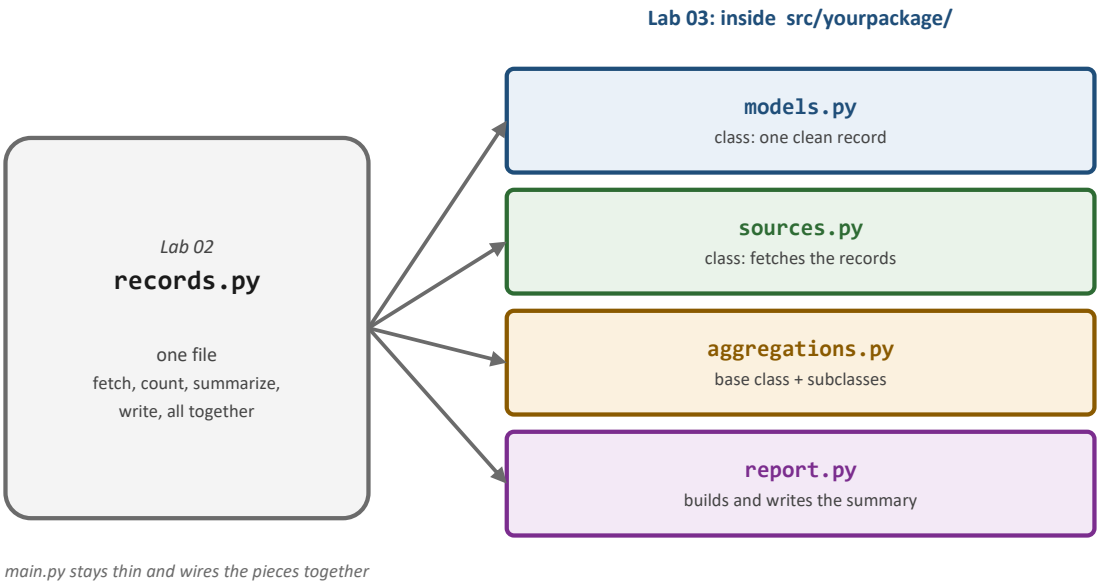


Figure 1. The Lab 02 file is split by responsibility. Your module names may differ; your package name should say what your program does.

You start from your own Lab 02 solution, whichever data source you chose. There is nothing new to download and no new data to learn. All of the effort goes into structure.

What your package must contain

Each of the following is checked directly during marking.

1. An installable package in a src layout, with a pyproject.toml, that installs with pip install -e . into a fresh conda environment and imports without any change to sys.path.
2. At least two classes, each with an __init__ method and a docstring. One class must represent a single record from your data (a show, a reading, an earthquake, an exchange rate) and must clean its own values when it is created, so that missing or malformed data is dealt with in one place. At least one class must define __str__.

3. Inheritance used for a real reason: a base class with at least two subclasses that override a method, and code that calls the overridden method on a mix of those objects without checking which subclass each one is. In most Lab 02 programs the aggregations are the natural fit: each one becomes a subclass of an Aggregation base class that overrides a compute method.
4. Modules organized by responsibility, with at least separate modules for the record class, for getting the data, for the aggregations, and for writing the output. Only the module that downloads the data may import requests. The package __init__.py re-exports the public names so that a user can write from yourpackage import ClassName.
5. Behaviour unchanged from Lab 02. The summary has the same keys and the same values for the same data, and a failed download still prints a clear message and exits without a traceback.
6. Settings in one place. The source URL, the timeout and the output path live in a single config module. No absolute file paths anywhere.
7. A conda environment file named environment.yml, and a requirements.txt with pinned versions, as described in the Environment files section below. pyproject.toml lists your dependencies.
8. A tests folder with at least two tests, in at least one file, that exercise one of your classes using hand-made records and no network. They must run with python -m unittest discover -s tests. Lab 04 will build this into a proper test suite.

Required layout

Name the package for what your program does, in lowercase with no hyphens or spaces: tvsummary, weathersummary, quakesummary. The names inside it are suggestions. The shape is not.

```
your-repository/
    README.md
    environment.yml          conda environment: name, Python version, pip
    requirements.txt         pinned pip packages
    pyproject.toml           tells pip where the package lives
    .gitignore
    main.py                  thin entry point; error message
    src/
        yourpackage/
            __init__.py      re-exports the public names
            config.py        URL, timeout, output path
            models.py        the record class
            sources.py       fetching; only module using requests
            aggregations.py  base class and subclasses
            report.py        build and write the summary
    tests/
        test_models.py
    data/
        raw/.gitkeep
        processed/summary.json
```

You may add more modules if your design calls for them. You may not put everything in one module and call it a package.

Suggested order of work

This is guidance, not a requirement, but it keeps a large change from turning into a large mess. Commit after each step.

1. Create the repository. Your first commit is your Lab 02 program, unchanged, so the history shows what you started from.

2. Create the conda environment and an empty package skeleton. Install it with pip install -e . and confirm that import yourpackage works before you move any code.
3. Write the record class. Move the checks for missing and malformed values out of your functions and into its __init__ method.
4. Move the download into a class in sources.py. It should return record objects, not raw dictionaries.
5. Turn your aggregation functions into subclasses of a base class. Change your summary builder to loop over a list of aggregation objects.
6. Move writing the summary into report.py, and reduce main.py to wiring things together and printing messages.
7. Run the program and compare the new summary.json to the one from Lab 02.
8. Write the tests, pin the requirements, and finish the README.

> **Check that nothing changed**
>
> Keep your Lab 02 summary.json. After refactoring, compare the two files, for example in VS Code by right-clicking one and choosing Select for Compare. For a stable source such as TV shows or a fixed exchange rate range, they should be identical.
>
> Weather and earthquake feeds change from run to run, so for those, check that the keys and structure match and that the logic still gives sensible numbers.

Environment files

Everyone on this course uses conda. Your repository must let someone else rebuild your environment without asking you a question. Two files do that, and they have different jobs.

environment.yml

This file names the environment and fixes the Python version. Keep it short: Python and pip, nothing else. The pip packages are pinned in requirements.txt.

```
name: yourpackage
channels:
  - defaults
dependencies:
  - python=3.11
  - pip
```

requirements.txt

Create your environment, install what you need with pip, and then write down exactly what is installed.

```
conda env create -f environment.yml
conda activate yourpackage
pip install requests
pip freeze > requirements.txt
pip install -e .
```

> **Watch out**

> Run pip freeze before pip install -e . If you freeze after the editable install, requirements.txt gains a line starting with -e and an absolute path from your own computer, which will fail on every other machine. If you already have that line, delete it.
>
> Do not submit the output of conda env export. It lists every package for your operating system and will not rebuild on a classmate's machine.

Repository requirements

Create a new repository for this lab. Do not add it to your Lab 01 or Lab 02 repository.

Item	Requirement
.gitignore	Committed before your first code commit, excluding __pycache__/, *.egg-info/ and .env. Do not ignore data/processed/ in this lab; the marker needs to see your output
README.md	Follows the template below, including the What moved where table and your design choices
environment.yml	Conda environment as described above
requirements.txt	Pinned versions, with no absolute paths
pyproject.toml	Package name, version and dependencies; src layout
src/ and main.py	The package and its entry point, as in the required layout
tests/	At least two passing tests that need no network
summary.json	The output of a successful run, committed under data/processed/
Commit history	At least five commits, each a coherent change with a message that says what changed. The first is your Lab 02 program, unchanged. The old single-file program is removed in the commit that replaces it

README template

Replace the content, not the shape. The What moved where table and the Design choices section carry marks in this lab.

```
# Project Name

One or two sentences on what this package summarizes.

## Data source
The URL, what one record represents, roughly how many records.

## Setup
conda env create -f environment.yml
conda activate yourpackage
pip install -r requirements.txt
pip install -e .

## Run
python main.py

## Test
python -m unittest discover -s tests
```

```
## Layout
One line for each module: what it contains and why it is a separate module.

## What moved where
A table with two columns: each function from your Lab 02 program,
and where that code lives now (class and method, or module and function).

## Design choices
Why each class exists. Where you used inheritance, what the base class
provides, and what adding one more subclass would involve.

## Known limitations
Anything that does not work, or that you would improve with more time.
```

How to submit

Post the URL of your GitHub repository to the Lab 03 submission on Blackboard. Do not upload files, and do not submit a link to a single file within the repository.

```
https://github.com/your-username/your-repository
```

Make sure the repository is public. If you prefer to keep it private, you must share it with the instructor account below. Sharing with any other account does not count as a submission and will be rejected.

> **Instructor GitHub account**
>
> hossein.pourmodheji@humber.ca
>
> Add this account as a collaborator under Settings, Collaborators in your repository. Any other account will be rejected.

A repository that a marker cannot open is treated as not submitted, and the late policy applies from the point at which access is granted.

> **Watch out**
>
> Before you submit, clone your repository into a different folder. Create the environment from environment.yml, activate it, run pip install -r requirements.txt and pip install -e ., then run the program and the tests following only your README.
>
> Then turn off your Wi-Fi and run the program again. It should print a clear message and stop. If you see a traceback, your error handling did not survive the move.

> **Looking ahead to Module 04**
>
> In Lab 04 you will add a full test suite to this package, replace print statements with logging, add type hints to your public functions, and exchange repositories with a classmate for a written code review. The cleaner your classes and modules are now, the less there is to untangle then.